
LET THEM COOK.

The complete playbook for Claude Code agent teams.
Spawn parallel agents. Coordinate real work. Ship faster.

THE MISSING MANUAL FOR MULTI-AGENT WORKFLOWS

One agent is a developer. A team of agents is an engineering org.
This guide shows you how to run the org.

MARCH 2026

Henry Nicholson

linkedin.com/in/henrymnicholson

CONTENTS

- 01 Why One Agent Isn't Enough
- 02 The Architecture: How Agent Teams Work
- 03 Subagents vs Agent Teams: When to Use What
- 04 Setting It Up
- 05 The Context Stack: What Every Agent Needs
- 06 The 5 Plays That Actually Work
- 07 Prompt: Parallel Feature Build
- 08 Prompt: Multi-Lens Code Review
- 09 Prompt: Competing Hypothesis Debugging
- 10 Prompt: Research & Exploration Sprint
- 11 Prompt: Full-Stack Refactor
- 12 Controlling the Team
- 13 What Will Go Wrong (And How to Fix It)
- 14 Token Economics: Is It Worth It?
- 15 The Engineering Manager Parallel
- 16 Start Cooking

WHY ONE AGENT ISN'T ENOUGH

You've been using Claude Code as a single agent. One session, one context window, one thread of execution. It works great for focused tasks. But the moment your project gets complex, that single agent starts to degrade.

The core problem: LLMs perform worse as context expands. The more information in the context window, the harder it is for the model to focus on what matters right now. Adding strategic planning notes to a context that's trying to fix a CSS bug actively hurts performance.

Human teams work the same way. You don't have backend engineers sitting in on frontend code reviews. Specialization is about focus.

Agent teams formalize this. By giving each agent a narrow scope and clean context, you get better reasoning within each domain, independent quality checks, natural checkpoints between phases, and graceful degradation when one agent gets stuck.

On February 5, 2026, Anthropic shipped agent teams as a first-class feature in Claude Code alongside Opus 4.6. This guide is the playbook.

THE ARCHITECTURE: HOW AGENT TEAMS WORK

One Claude Code session becomes the team lead. It spawns teammates, each a full, independent Claude Code instance with its own context window.

THE FOUR COMPONENTS

- **Team Lead:** Your main session. Creates the team, spawns teammates, assigns tasks, synthesizes results.
- **Teammates:** Separate Claude instances. Each gets their own context window, loads project context (CLAUDE.md, MCP servers), and works independently.
- **Shared Task List:** Central work items with dependency tracking. Blocked work auto-unblocks when dependencies finish.
- **Mailbox:** Direct messaging between agents. Teammates message each other, not just the lead.

HOW SPAWNING WORKS

You describe the team in natural language. Each teammate loads your CLAUDE.md but does NOT inherit the lead's conversation history. The spawn prompt is the only briefing they get.

The spawn prompt is everything. Make it specific. Include which files they own, what patterns to follow, and what done looks like.

SUBAGENTS VS AGENT TEAMS: WHEN TO USE WHAT

The deciding question: do your workers need to communicate with each other?

USE SUBAGENTS WHEN:

- You need quick, focused workers that report back results
- Tasks are independent, no cross-agent communication needed
- You want lower token cost

USE AGENT TEAMS WHEN:

- Teammates need to share findings and challenge each other
- Work spans multiple domains that need to stay coordinated
- The problem benefits from parallel exploration with cross-pollination

USE NEITHER WHEN:

- The task is sequential, same-file edits, or simple enough for one session
- You're fixing a typo. Don't spawn a swarm for a typo.

SETTING IT UP

STEP 1: ENABLE THE FLAG

```
# Environment variable export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 # Or settings.json
{ "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" } }
```

STEP 2: CHOOSE DISPLAY MODE

- **In-process (default):** All teammates in your terminal. Shift+Up/Down to select.
- **tmux split panes:** Each teammate gets its own pane. Requires tmux or iTerm2.

STEP 3: SET UP YOUR CLAUDE.MD

Every teammate reads it on startup. Include tech stack, conventions, file structure, and coding patterns.

THE CONTEXT STACK: WHAT EVERY AGENT NEEDS

Each agent gets context from three layers.

LAYER 1: CLAUDE.MD (PROJECT-LEVEL)

Auto-read on startup. Tech stack, conventions, project rules. The shared foundation.

LAYER 2: CONTEXT FILES (DOMAIN-LEVEL)

Brand voice profiles, API specs, data models. Each agent loads only what's relevant to their domain.

LAYER 3: SPAWN PROMPT (TASK-LEVEL)

The briefing defining role, file ownership, deliverable, and constraints.

Layer 1 keeps everyone aligned. Layer 2 gives domain knowledge. Layer 3 gives each agent their mission. Miss any layer and the team underperforms.

THE 5 PLAYS THAT ACTUALLY WORK

PLAY 1: PARALLEL FEATURE BUILD

Split a new feature across frontend, backend, and tests. Each agent owns a layer.

PLAY 2: MULTI-LENS CODE REVIEW

Spawn 3-4 reviewers: security, performance, test coverage, maintainability.

PLAY 3: COMPETING HYPOTHESIS DEBUGGING

Spawn agents investigating different theories. They challenge each other like a scientific debate.

PLAY 4: RESEARCH & EXPLORATION SPRINT

Multiple agents investigate different approaches and converge on a recommendation.

PLAY 5: FULL-STACK REFACTOR

One agent per module. They work in parallel and validate interfaces align.

COPY-PASTE PROMPT: PARALLEL FEATURE BUILD

The workhorse play.

PROMPT

```
Create an agent team to build [FEATURE NAME]. Reference the project CLAUDE.md for all
conventions. Spawn 3 teammates: 1. Backend Agent: Owns /src/server/[feature]/. Build
the API endpoints and data layer. Deliverable: working endpoints with types exported.
2. Frontend Agent: Owns /src/client/[feature]/. Build the React components and state
management. Wait for API types from Backend before building data fetching layer.
Deliverable: working UI connected to API. 3. Test Agent: Owns /tests/[feature]/. Write
unit and integration tests as each piece lands. Deliverable: passing test suite with
coverage. Coordinate through the shared task list. Use delegate mode -- lead
coordinates only, does not implement. [DESCRIBE THE FEATURE SPEC]
```

COPY-PASTE PROMPT: MULTI-LENS CODE REVIEW

Use before merging significant changes.

PROMPT

```
Create an agent team to review [BRANCH/PR/FILES]. Spawn 4 teammates: 1. Security
Reviewer: Audit for vulnerabilities. Rate findings by severity. 2. Performance
Reviewer: Profile for bottlenecks. Suggest specific optimizations. 3. Test Coverage
Reviewer: Identify untested paths. Write the missing tests if under 80% coverage. 4.
Architecture Reviewer: Check maintainability. Evaluate patterns, naming, separation of
concerns. Have them share findings and challenge each other. Compile a unified review
report at the end. [DESCRIBE WHAT CHANGED AND WHY]
```

COPY-PASTE PROMPT: COMPETING HYPOTHESIS DEBUGGING

Use when something is broken and you don't know why.

PROMPT

```
Something is broken: [DESCRIBE THE BUG] Spawn 4 teammates, each investigating a
different hypothesis: 1. Agent A: Investigate [hypothesis 1] 2. Agent B: Investigate
[hypothesis 2] 3. Agent C: Investigate [hypothesis 3] 4. Agent D: Investigate
[hypothesis 4] Rules: - Each agent should try to PROVE their hypothesis - Share
findings with other agents - Challenge each other's theories - The lead synthesizes all
findings into a root cause analysis with evidence for/against each hypothesis [PASTE
ERROR LOGS OR REPRODUCTION STEPS]
```

COPY-PASTE PROMPT: RESEARCH & EXPLORATION SPRINT

Use when evaluating multiple approaches.

PROMPT

```
I need to decide: [DESCRIBE THE DECISION] Spawn 3 teammates: 1. Option A Advocate: Research and build the case for [option A]. Concrete evidence only. 2. Option B Advocate: Same depth for [option B]. 3. Devil's Advocate: Challenge both options. Find weaknesses. Propose Option C if needed. The lead compiles a decision matrix with: - Comparison across key criteria - Tradeoffs for each option - Clear recommendation with reasoning [DESCRIBE YOUR CONSTRAINTS]
```

COPY-PASTE PROMPT: FULL-STACK REFACTOR

Use for large refactors touching multiple modules.

PROMPT

```
Refactor: [DESCRIBE WHAT AND WHY] Assign directory-level ownership: 1. Agent 1: Owns [/src/module-a/]. Refactor to [new pattern]. 2. Agent 2: Owns [/src/module-b/]. Coordinate with Agent 1 on shared interfaces. 3. Agent 3: Owns [/src/module-c/]. Update imports from modules A and B. 4. Test Agent: Owns /tests/. Run existing tests continuously. Flag any regressions immediately. Rules: - NO agent edits files outside their owned directory - When an interface changes, message all dependents - Lead requires plan approval before breaking changes [DESCRIBE CURRENT STATE AND TARGET STATE]
```

CONTROLLING THE TEAM

KEYBOARD CONTROLS

- **Shift+Up/Down:** Select a teammate to message directly
- **Enter:** View a teammate's session
- **Escape:** Interrupt a teammate's current turn
- **Ctrl+T:** Toggle the shared task list view
- **Shift+Tab:** Toggle delegate mode (lead coordinates only)

DELEGATE MODE

Restricts the lead to coordination only. Prevents it from implementing instead of waiting for teammates. Use for any team larger than 2.

PLAN APPROVAL

Require teammates to plan before implementing. Read-only mode until the lead approves.

SHUTTING DOWN

Always use the lead to clean up. Shut down all teammates before cleaning up the team.

WHAT WILL GO WRONG (AND HOW TO FIX IT)

THE LEAD IMPLEMENTS INSTEAD OF DELEGATING

Fix: use delegate mode (Shift+Tab) or tell the lead 'Wait for your teammates.'

TASK STATUS GETS STALE

Teammates forget to mark tasks completed. Fix: check manually and nudge the lead.

TWO AGENTS EDIT THE SAME FILE

Merge conflict chaos. Fix: assign clear directory-level ownership in the spawn prompt.

AGENTS LOSE FOCUS

Fix: break into smaller sub-tasks. 5-6 tasks per teammate is the sweet spot.

NO SESSION RESUMPTION

Can't resume in-process teammates after disconnect. Fix: spawn fresh teammates.

RUNAWAY TOKEN COSTS

Each teammate is a full Claude instance. Fix: only use when parallel exploration adds real value.

TOKEN ECONOMICS: IS IT WORTH IT?

Agent teams always cost more. The question is whether the output justifies it.

WHEN IT'S WORTH IT

- Task would take 4+ hours single-session. Parallel cuts it to 1-2 hours.
- Quality improves from focused context per agent.
- You catch issues a single reviewer would miss.

WHEN IT'S NOT WORTH IT

- Simple sequential tasks, same-file edits, or anything under 30 minutes.

Think of it like hiring contractors. You wouldn't hire 4 plumbers to fix one sink. But you would hire 4 specialists to renovate a house in parallel.

THE ENGINEERING MANAGER PARALLEL

The skills that make a strong EM translate directly into effective agent orchestration.

TASK SIZING

5-6 tasks per teammate. Too small = coordination overhead. Too large = drift.

FILE OWNERSHIP

Each teammate owns different files. Same as avoiding merge conflicts with humans.

CONTEXT LOADING

Specific briefings produce better output. You wouldn't tell a new hire 'just figure it out.'

KNOWING WHEN TO INTERVENE

Set direction, check milestones, course-correct when needed. Don't micromanage.

KNOWING WHEN TO KILL IT

If an agent is going in circles, shut it down and respawn with a clearer prompt.

START COOKING

Don't start with a 5-agent feature build. Start small. Build intuition. Then scale.

WEEK 1: CODE REVIEW

Spawn a 3-agent review team. Read-only. No risk. See how they coordinate.

WEEK 2: RESEARCH SPRINT

Spawn advocates + devil's advocate for a real decision you're facing.

WEEK 3: DEBUGGING

Spawn agents with competing hypotheses. Watch parallel investigation beat sequential.

WEEK 4: FEATURE BUILD

Full team: backend, frontend, tests. Delegate mode. Let them cook.

One agent is a developer. A team of agents is an engineering org. The question isn't whether to use them. It's whether you're ready to manage them.

Set the context. Define the roles. Assign the ownership. Then let them cook.

If you made it this far, comment "cook" on the post. I want to know who actually reads these.

LET THEM COOK · MARCH 2026

Henry Nicholson

Co-Founder, ForeFront USD

linkedin.com/in/henrymnicholson